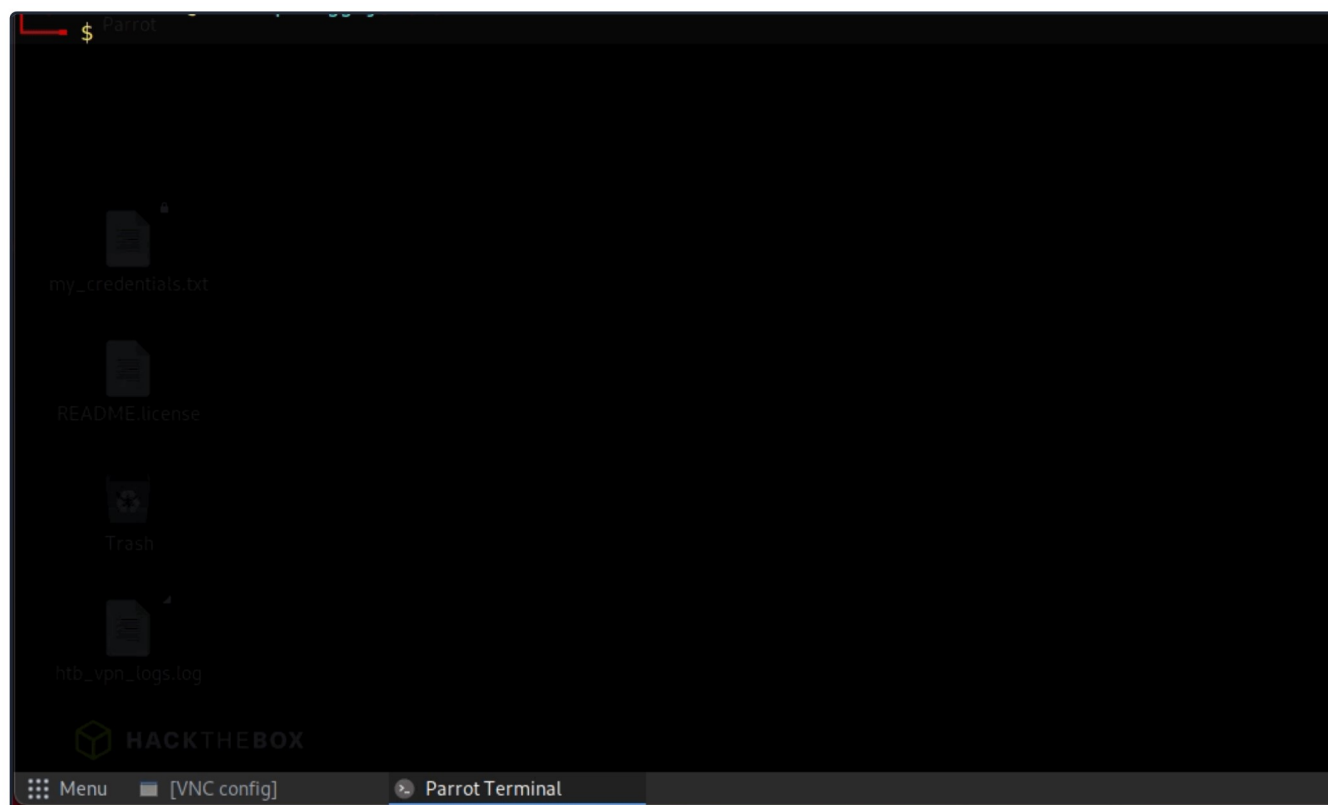


Shells Jack Us In, Payloads Deliver Us Shells



A **shell** is a program that provides a computer user with an interface to input instructions into the system and view text output (Bash, Zsh, cmd, and PowerShell, for example). As penetration testers and information security professionals, a shell is often the result of exploiting a vulnerability or bypassing security measures to gain interactive access to a host. We may have heard or read the following phrases used by people discussing an engagement or a recent practice session:

- "I caught a shell."
- "I popped a shell!"
- "I dropped into a shell!"
- "I'm in!"

Typically these phrases translate to the understanding that this person has successfully exploited a vulnerability on a system and has been able to gain remote control of the shell on the target computer's operating system. This is a common goal a penetration tester will have when attempting to access a vulnerable machine. We will notice that most of this module will focus on what comes after enumeration and identification of promising exploits.

Why Get a Shell?

Remember that the shell gives us direct access to the **OS**, **system commands**, and **file system**. So if we gain access, we can start enumerating the system for vectors that may allow us to escalate privileges, pivot, transfer files, and more. If we don't establish a shell session, we are pretty limited on how far we can get on a target machine.

Establishing a shell also allows us to maintain persistence on the system, giving us more time to work. It can make it easier to use our **attack tools**, **exfiltrate data**, **gather**, **store** and **document** all the details of our attack, as we will soon see in the proceeding demonstrations. It's important to note that establishing a shell almost always means we are accessing the CLI of the OS, and this can

make us harder to notice than if we were remotely accessing a graphical shell over [VNC](#) or [RDP](#). Another significant benefit of becoming skilled with command-line interfaces is that they can be **harder to detect than graphical shells**, **faster to navigate the OS**, and **easier to automate our actions**. We view shells through the lens of the following perspectives throughout this module:

Perspective	Description
Computing	The text-based userland environment that is utilized to administer tasks and submit instructions on a PC. Think Bash, Zsh, cmd, and PowerShell.
Exploitation & Security	A shell is often the result of exploiting a vulnerability or bypassing security measures to gain interactive access to a host. An example would be triggering EternalBlue on a Windows host to gain access to the cmd-prompt on a host remotely.
Web	This is a bit different. A web shell is much like a standard shell, except it exploits a vulnerability (often the ability to upload a file or script) that provides the attacker with a way to issue instructions, read and access files, and potentially perform destructive actions to the underlying host. Control of the web shell is often done by calling the script within a browser window.

Payloads Deliver us Shells

Within the IT industry as a whole, a **payload** can be defined in a few different ways:

- **Networking**: The encapsulated data portion of a packet traversing modern computer networks.
- **Basic Computing**: A payload is the portion of an instruction set that defines the action to be taken. Headers and protocol information removed.
- **Programming**: The data portion referenced or carried by the programming language instruction.
- **Exploitation & Security**: A payload is **code** crafted with the intent to exploit a vulnerability on a computer system. The term payload can describe various types of malware, including but not limited to ransomware.

In this module, we will be working with many different types of **payloads** and delivery methods within the context of granting ourselves access to a host and establishing **remote shell** sessions with vulnerable systems.

[Next →](#)[✓ Mark Complete & Next](#)[📄 Cheat Sheet](#)

Table of Contents

Introduction

[Shells Jack Us In, Payloads Deliver Us Shells](#)[CAT5 Security's Engagement Preparation](#)

Shell Basics

[📁 Anatomy of a Shell](#)[📁 Bind Shells](#)[📁 Reverse Shells](#)

Payloads

[Introduction to Payloads](#)[📁 Automating Payloads & Delivery with Metasploit](#)[Crafting payloads with MSFvenom](#)

0 / 0

Windows Shells

 Infiltrating Windows

NIX Shells

 Infiltrating Unix/Linux

Spawning Interactive Shells

Web Shells

Introduction to Web Shells

 Laudanum, One Webshell to Rule Them All Antak Webshell PHP Web Shells

Skills Assessment

 The Live Engagement

Additional Considerations

Detection & Prevention

My Workstation

OFFLINE

 Start Instance / 1 spawns left